FORMATION « PYTHON EN 24 HEURES » | SÉANCE 06

Organiser ses données et son code : dict, set et fonctions

Chapitres 11 et 12 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Créer et manipuler des dictionnaires : clés, valeurs, get et .items()
- Compter et regrouper des données avec un dictionnaire
- Utiliser les ensembles pour l'unicité et les opérations ensemblistes
- Écrire des fonctions avec paramètres, valeurs par défaut et return
- Documenter une fonction avec une docstring et choisir la bonne structure

Rappel de la séance 05

Les listes rangent par position. Quand la position n'a pas de sens, il faut une autre organisation : l'accès par clé.

- Index, slices, méthodes append, insert, remove, pop
- Tuples : immuables, parfaits pour retourner plusieurs valeurs
- Compréhensions : simple, filtrée, imbriquée
- Piège : affecter une liste ne la copie pas
- enumerate pour parcourir avec la position

Le dictionnaire : accès par clé

```
eleve = {"nom": "Awa", "age": 21}
print(eleve["nom"])            # Awa
eleve["filiere"] = "Maths"     # ajout
eleve["age"] = 22              # modification
print("nom" in eleve)          # True
print(eleve.get("tel"))        # None : pas d'erreur !
```

Clé entre crochets : lecture et écriture. get(key) renvoie None au lieu de lever KeyError quand la clé manque.

Parcourir un dictionnaire

- ‘.keys()‘ : les clés (le parcours direct fait pareil)
- ‘.values()‘ : les valeurs seules
- ‘.items()‘ : les paires (clé, valeur), dépliées par l’unpacking
- Les dictionnaires conservent l’ordre d’insertion depuis Python 3.7
- Les clés sont uniques : réaffecter remplace la valeur

```python
stocks = {"riz": 50, "huile": 12}

for cle in stocks:
    print(cle, stocks[cle])

for cle, valeur in stocks.items():
    print(f"{cle:<8} : {valeur:>3} sacs")
```

Compter avec un dictionnaire

```
mots = ["riz", "mais", "riz", "yam", "mais", "riz"]
compteur = {}
for mot in mots:
    if mot in compteur:
        compteur[mot] += 1
    else:
        compteur[mot] = 1
print(compteur)
```

Le schéma de comptage : si la clé existe, incrémenter ; sinon, créer à 1. La méthode setdefault ou .get(k, 0) + 1 condense ce code.

L'ensemble : unicité garantie

- Création : ‘set([1, 2, 2, 3])‘ donne {1, 2, 3}
- Aucun doublon, aucun ordre garanti
- ‘in‘ est **très rapide**, même sur de gros ensembles
- union ‘|‘, intersection ‘&‘, différence ‘-‘
- Usage typique : dédoublonner une liste

```
inscrits = ["awa", "kofi", "awa", "sena"]
uniques = set(inscrits)
print(uniques)

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)   # {1, 2, 3, 4}
print(a & b)   # {2, 3}
print(a - b)   # {1}
```

Quelle structure choisir ?

Structure	Quand l'utiliser	Exemple
list	série ordonnée, modifiable	file d'attente, notes
tuple	groupe figé de valeurs	coordonnées, retour multiple
dict	accès par clé signifiante	fiche élève, inventaire
set	unicité et appartenance rapide	dédoublonner, tags

Règle pratique : la position compte ? list. Le nom compte ? dict. Le doublon interdit ? set.

Définir une fonction

```python
def aire_rectangle(longueur, largeur):
    """Retourne l'aire d'un rectangle."""
    return longueur * largeur

resultat = aire_rectangle(8, 5)
print(resultat)   # 40
```

def lance la définition, la docstring documente, return rend la valeur. Une fonction sans return rend None.

Paramètres : ordre, défauts, mots clés

- L'ordre des arguments positionnels compte
- Un paramètre avec défaut devient **facultatif**
- Les arguments par mots clés rendent l'appel explicite
- Les valeurs par défaut sont évaluées **une seule fois**
- Jamais de liste mutable en défaut (piège du Tome 1)

```python
def prix_ttc(hors_taxe, taux=0.18, remise=0):
    net = hors_taxe * (1 + taux)
    return round(net * (1 - remise), 2)

print(prix_ttc(100))
print(prix_ttc(100, taux=0.075))
print(prix_ttc(100, remise=0.1, taux=0.075))
```

Portée : locale par défaut

```python
total_global = 1000      # variable globale

def ajouter(montant):
    part_locale = montant       # locale à la fonction
    return part_locale

print(ajouter(50))
print(total_global)
# print(part_locale)  -> NameError : hors de la fonction
```

Une variable créée dans une fonction disparaît à la sortie. Une fonction lit les globales mais doit les éviter : passez tout en paramètres.

Retourner plusieurs valeurs

```python
def min_max_moy(notes):
    """Retourne (minimum, maximum, moyenne) d'une série."""
    return (min(notes), max(notes),
            round(sum(notes) / len(notes), 2))

mini, maxi, moy = min_max_moy([12, 15, 9, 18])
print(mini, maxi, moy)
```

Le tuple retourné se déplie en trois variables à l'appel : c'est le idiome standard du calcul de statistiques.

Attention

KeyError : la clé fantôme

‘eleve["tel"]‘ lève ‘KeyError‘ si la clé n’existe pas. Deux parades : ‘.get("tel")‘ qui renvoie None, ou le test ‘"tel" in eleve‘ avant l’accès. Choisissez selon l’intention : get quand l’absence est normale, in quand l’absence est une anomalie à signaler.

À retenir

L'essentiel de la séance 06

Le dictionnaire relie une clé à une valeur et se parcourt avec .items() ; l'ensemble garantit l'unicité et teste l'appartenance en un éclair. La structure se choisit selon la question posée : position, nom ou unicité. La fonction encapsule un calcul derrière un nom clair, documenté par une docstring, avec des paramètres explicites : c'est la brique de base de tout gros programme.

fonctions

Cinq exercices pour organiser les données puis le code : fiche contact, comptage, validation, statistiques et un mini quiz complet. Les fonctions des exercices 3 à 5 doivent avoir une docstring d'une ligne.

- **Exercice 1 (Basique)** : Fiche contact
- **Exercice 2 (Basique)** : Compteur d'occurrences
- **Exercice 3 (Intermédiaire)** : Validation d'adresse e-mail
- **Exercice 4 (Intermédiaire)** : Statistiques de notes en dictionnaire
- **Exercice 5 (Défi)** : Mini quiz en fonctions

Ouvrez le notebook « Seance_06_dict_set_fonctions.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 07 : fonctions avancées (closures et décorateurs), puis chaînes et fichiers. Relisez la portée locale et globale de cette séance : la closure est exactement une fonction qui garde en mémoire la portée où elle est née.